



UNIVERSITY OF MORATUWA
Department of Electronic & Telecommunication Engineering

EN2111: Electronic Circuit Design

UART Implementation in FPGA

Deshan W.U.	230130E
Dhananjaya K.T.G.T.N	230138K
Dharmakeerthi P.K.G.C.L.	230140J

April 24, 2026

Contents

1	Introduction	2
2	UART Design	2
2.1	Code Explanation	2
2.2	RTL Code	2
2.2.1	UART Transmitter	2
2.2.2	UART Receiver	4
2.2.3	Seven segment Module	5
2.2.4	UART Top Module	6
2.2.5	UART Testbench	7
3	Simulation Results	9
4	Hardware Implementation and Testing	10
5	Conclusion	10

1 Introduction

Universal Asynchronous Receiver/Transmitter (UART) is a widely used serial communication protocol that enables data transfer between digital devices without requiring a shared clock signal. It operates asynchronously, meaning that both transmitter and receiver agree on a predefined baud rate to correctly interpret data.

In this assignment, the objective is to implement a UART transceiver using Verilog HDL and deploy the design on an FPGA platform. The system converts parallel data into serial form at the transmitter and reconstructs the original parallel data at the receiver.

The project involves developing a Register Transfer Level (RTL) design for UART, creating a testbench for functional verification, and implementing the design on FPGA hardware. The communication is verified by interfacing with another FPGA system and displaying the transmitted data using seven-segment displays.

Additionally, simulation waveforms and oscilloscope measurements are used to analyze timing behavior and validate correct UART operation. This report presents the design, implementation, simulation results, and hardware verification of the UART system.

2 UART Design

2.1 Code Explanation

The UART receiver operates using a finite state machine with two main states: IDLE and READ. In the IDLE state, the line remains high. When a start bit (logic 0) is detected, the system transitions to the READ state.

A tick signal, generated at 16 times the baud rate, is used to sample incoming data bits at the midpoint, ensuring reliable data reception.

2.2 RTL Code

2.2.1 UART Transmitter

```
1 module uart_tx #(
2     parameter CLOCKS_PER_PULSE = 5208    // clock(50MHz) & baud
3     (9600)
4 )(
5     input  logic clk,
6     input  logic rstn,
7     input  logic [7:0] data_in,
8     input  logic data_en,
9     output logic tx,
10    output logic tx_busy
11 );
12
13 // Enum for states
14 typedef enum logic [1:0] {IDLE, START, DATA, STOP} state_t;
15 state_t state;
16
17 logic [7:0] data;
18 logic [2:0] bit_index;
```

```

18     logic [$clog2(CLOCKS_PER_PULSE)-1:0] clk_count;
19
20     always_ff @(posedge clk or negedge rstn) begin
21         if (!rstn) begin
22             state <= IDLE;
23             tx <= 1'b1;
24             clk_count <= 0;
25             bit_index <= 0;
26         end else begin
27             case (state)
28
29                 // IDLE state (Wait until data_en button is pressed)
30                 IDLE: begin
31                     tx <= 1'b1;
32                     if (data_en) begin
33                         data <= data_in;
34                         state <= START;
35                         clk_count <= 0;
36                     end
37                 end
38
39                 // START state (Send the start bit '0')
40                 START: begin
41                     tx <= 1'b0;
42                     if (clk_count == CLOCKS_PER_PULSE-1) begin
43                         clk_count <= 0;
44                         bit_index <= 0;
45                         state <= DATA;
46                     end else clk_count <= clk_count + 1;
47                 end
48
49                 // DATA state (Send the data)
50                 DATA: begin
51                     tx <= data[bit_index];
52                     if (clk_count == CLOCKS_PER_PULSE-1) begin
53                         clk_count <= 0;
54                         if (bit_index == 7)
55                             state <= STOP;
56                         else
57                             bit_index <= bit_index + 1;
58                     end else clk_count <= clk_count + 1;
59                 end
60
61                 // STOP state (Send the stop bit '1' and go to IDLE)
62                 STOP: begin
63                     tx <= 1'b1;
64                     if (clk_count == CLOCKS_PER_PULSE-1) begin
65                         state <= IDLE;
66                         clk_count <= 0;
67                     end else clk_count <= clk_count + 1;
68

```

```

69             end
70         endcase
71     end
72 end
73 end
74
75     assign tx_busy = (state != IDLE);
76
77 endmodule

```

2.2.2 UART Receiver

```

1 module uart_rx #(
2     parameter CLOCKS_PER_PULSE = 5208
3 ) (
4     input  logic clk,
5     input  logic rstn,
6     input  logic rx,
7     output logic [7:0] data_out,
8     output logic data_valid
9 );
10
11 // Enum for states
12 typedef enum logic [1:0] {IDLE, START, DATA, STOP} state_t;
13 state_t state;
14
15 logic [7:0] data;
16 logic [2:0] bit_index;
17 logic [$clog2(CLOCKS_PER_PULSE)-1:0] clk_count;
18
19 always_ff @(posedge clk or negedge rstn) begin
20     if (!rstn) begin
21         state <= IDLE;
22         clk_count <= 0;
23         bit_index <= 0;
24         data_valid <= 0;
25     end else begin
26         case (state)
27
28             // IDLE state (Wait until the start bit '0' is
29             // detected)
30             IDLE: begin
31                 data_valid <= 0;
32                 if (rx == 0) begin // start bit detected
33                     state <= START;
34                     clk_count <= 0;
35                 end
36             end
37
38             // START state

```

```

38         START: begin
39             if (clk_count == (CLOCKS_PER_PULSE/2)) begin
40                 clk_count <= 0;
41                 bit_index <= 0;
42                 state <= DATA;
43             end else clk_count <= clk_count + 1;
44         end
45
46         // DATA state (Read the 8 bit data to data[ ] )
47         DATA: begin
48             if (clk_count == CLOCKS_PER_PULSE-1) begin
49                 clk_count <= 0;
50                 data[bit_index] <= rx;
51
52                 if (bit_index == 7)
53                     state <= STOP;
54                 else
55                     bit_index <= bit_index + 1;
56
57             end else clk_count <= clk_count + 1;
58         end
59
60         // STOP state
61         STOP: begin
62             if (clk_count == CLOCKS_PER_PULSE-1) begin
63                 data_out <= data;
64                 data_valid <= 1;
65                 state <= IDLE;
66                 clk_count <= 0;
67             end else clk_count <= clk_count + 1;
68         end
69
70     endcase
71 end
72 end
73
74 endmodule

```

2.2.3 Seven segment Module

```

1 module seven_seg (
2     input  logic [3:0] bin,
3     output logic [6:0] seg
4 );
5
6     always_comb begin
7         case (bin)
8             4'h0: seg = 7'b1000000;
9             4'h1: seg = 7'b1111001;
10            4'h2: seg = 7'b0100100;

```

```

11         4'h3: seg = 7'b0110000;
12         4'h4: seg = 7'b0011001;
13         4'h5: seg = 7'b0010010;
14         4'h6: seg = 7'b0000010;
15         4'h7: seg = 7'b1111000;
16         4'h8: seg = 7'b0000000;
17         4'h9: seg = 7'b0010000;
18         4'hA: seg = 7'b0001000;
19         4'hB: seg = 7'b0000011;
20         4'hC: seg = 7'b1000110;
21         4'hD: seg = 7'b0100001;
22         4'hE: seg = 7'b0000110;
23         4'hF: seg = 7'b0001110;
24     endcase
25 end
26
27 endmodule

```

2.2.4 UART Top Module

```

1 module uart_top (
2     input  logic clk,
3     input  logic rstn,
4     input  logic [3:0] sw,
5     input  logic send,
6     input  logic rx,
7
8     output logic tx,
9     output logic [7:0] led,
10    output logic [6:0] seg
11 );
12
13    logic [7:0] received_data;    // Output from RX
14    logic [7:0] rx_value;        // Stores last received byte
15    logic send_prev;            // Used for edge detection
16    logic tx_send_pulse;
17    logic data_valid;           // RX finished signal
18
19    assign tx_send_pulse = send & ~send_prev;    // (1 and ~0) =>
    pulse; make sure that Only one transmission per press
20
21    always_ff @(posedge clk or negedge rstn) begin
22        if (!rstn) begin
23            rx_value <= 8'h00;
24            send_prev <= 1'b0;
25        end else begin
26            send_prev <= send;
27            if (data_valid)
28                rx_value <= received_data;
29        end

```

```

30     end
31
32
33     // Instantiate RX
34     uart_rx rx_inst (
35         .clk(clk),
36         .rstn(rstn),
37         .rx(rx),
38         .data_out(received_data),
39         .data_valid(data_valid)
40     );
41
42
43     // Instantiate TX
44     uart_tx tx_inst (
45         .clk(clk),
46         .rstn(rstn),
47         .data_in({4'b0000,sw}),
48         .data_en(tx_send_pulse),
49         .tx(tx),
50         .tx_busy()
51     );
52
53
54     // Display received(last) byte on seven-segment.
55     seven_seg seven_seg_inst (
56         .bin(rx_value[3:0]),
57         .seg(seg)
58     );
59
60     // Show the last valid RX byte on LEDs.
61     assign led = rx_value;
62
63     endmodule

```

2.2.5 UART Testbench

```

1  'timescale 1ns/1ps      //All delays like #10, #50 are in
   nanoseconds
2
3  module tb_uart;
4
5      logic clk;
6      logic rstn;
7      logic data_en;      // trigger transmission (changed from
   send)
8      logic [7:0] data_in; // data to send
9      logic tx;
10     logic tx_busy;      // TX status
11

```

```

12     logic [7:0] data_out;    // received byte
13     logic data_valid;       // indicates valid data
14
15     // Clock generation (50 MHz => 20 ns period)
16     always #10 clk = ~clk;
17
18     // TX
19     uart_tx tx_inst (
20         .clk(clk),
21         .rstn(rstn),
22         .data_en(data_en),
23         .data_in(data_in),
24         .tx(tx),
25         .tx_busy(tx_busy)
26     );
27
28     // RX
29     uart_rx rx_inst (
30         .clk(clk),
31         .rstn(rstn),
32         .rx(tx),             // TX output is directly connected to RX
                               input
33         .data_out(data_out),
34         .data_valid(data_valid)
35     );
36
37     initial begin           // This runs once at the start of simulation
38         clk = 0;
39         rstn = 0;
40         data_en = 0;
41
42         // First byte
43         data_in = 8'b10100101;
44
45         // Reset
46         #50;
47         rstn = 1;
48
49         #100;
50         data_en = 1;
51         #20;
52         data_en = 0;
53
54         // Wait until transmission finishes
55         wait (tx_busy == 0);
56
57         // wait for RX to finish
58         #10000;
59
60
61         // Second byte

```

```

62         data_in = 8'b11001100;
63
64         #100;
65         data_en = 1;
66         #20;
67         data_en = 0;
68
69         wait (tx_busy == 0);
70
71         #10000;
72
73         $stop;
74     end
75
76 endmodule

```

3 Simulation Results

Simulation waveforms were generated to verify the correctness of the UART protocol implementation.

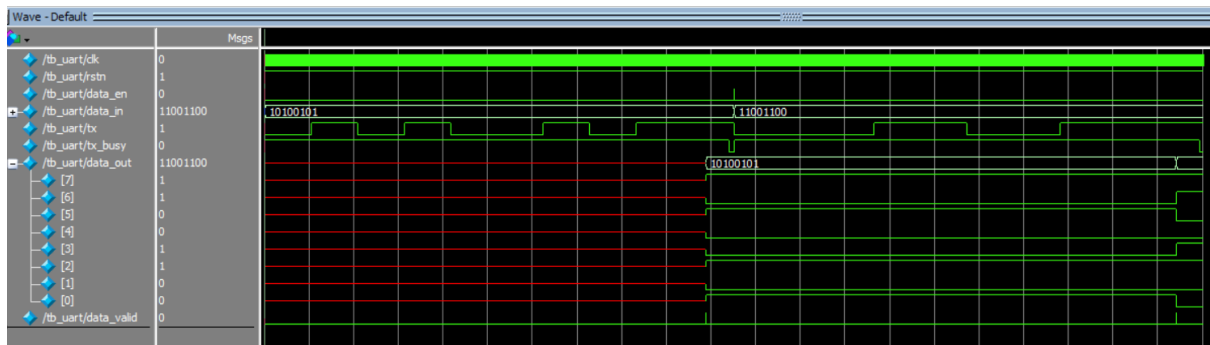


Figure 1: UART simulation waveforms showing transmission and reception

Two bytes were transmitted sequentially, with each transmission initiated when the data_en signal became high. At the beginning of the simulation, the data_out signals remained uninitialized (indicated by red lines), due to the absence of valid received data. Once the receiver successfully captured all the bits of the transmitted byte, the data_out bus was updated with the reconstructed data.

4 Hardware Implementation and Testing

The UART design was implemented on an FPGA and tested successfully.

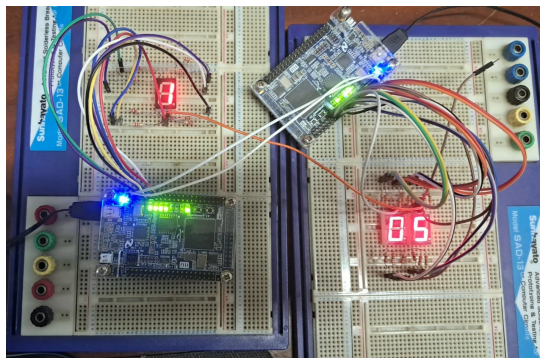


Figure 2: FPGA hardware setup and testing results

The onboard four DIP switches were used to generate the lower four bits of the transmitted byte, while the upper four bits were fixed at zero. The transmitted signal was captured by another FPGA board and displayed on a seven-segment display. Furthermore, data transmitted from the other FPGA board was also received correctly and displayed on the seven-segment display confirming the bidirectional communication between two systems.

5 Conclusion

This project successfully demonstrated the design and implementation of a UART transceiver using Verilog HDL on an FPGA platform. The developed RTL design was verified through simulation, ensuring correct data transmission and reception according to UART protocol specifications.

The hardware implementation further validated the functionality of the system by enabling reliable communication between two FPGA boards. The use of a baud rate generator and oversampling technique ensured accurate timing and robust data recovery in an asynchronous environment.

Simulation results and hardware observations confirmed the correct formation of start bits, data bits, and stop bits, as well as proper synchronization between transmitter and receiver.

Overall, this assignment provided valuable hands-on experience in digital system design, hardware description languages, and FPGA-based implementation of communication protocols. It also enhanced understanding of timing constraints, verification techniques, and real-world hardware debugging.

References

- [1] DE0-Nano SoC User Manual.
Available: https://www.terasic.com.tw/attachment/archive/941/DE0-Nano-SoC_User_manual_rev.D0.pdf

- [2] Serial Communication (TX and RX) Basics.
Available: <https://learn.sparkfun.com/tutorials/serial-communication/all>